Reproducible and Interoperable Recommendation Datasets and Data Processing with DataRec ⋆

Alberto Carlo Maria Mancino[1], Salvatore Bufi[2,†], Angela Di Fazio[1], Antonio Ferrara[1], Daniele Malitesta[3], Claudio Pomo[1] and Tommaso Di Noia[1]

[1]Politecnico di Bari, Bari, Italy
[2]Independent Researcher
[3]LUISS Guido Carli University, Rome, Italy

Abstract

Experimental results in recommender systems are strongly affected by decisions made before model training: the dataset version, interaction filtering, and train–test split. Yet these choices are often implemented in framework-specific and insufficiently documented ways, obstructing reproducibility and fair comparison. We present DataRec, an open-source Python library that makes recommendation-data management explicit, traceable, and reusable across experimental pipelines. The library offers version-aware access to widely used datasets, a unified representation of interactions, configurable processing and splitting operations, and export utilities for established recommendation frameworks. Its design is informed by an analysis of 55 recent papers and a comparison of data-management support in popular frameworks. By recording every operation, its parameters, random seeds, and checksums, DataRec turns a preprocessing pipeline into a reproducible artifact. This extended abstract summarizes the motivation, design, and contribution of the SIGIR 2025 DataRec paper.

Keywords

Recommender systems, reproducibility, data management, interoperability, Python library

1. Introduction

Recommender systems support decisions in domains such as e-commerce, entertainment, health, and education [2, 3, 4]. Their progress relies on empirical evaluation, but reliable comparison of results requires more than sharing model code. Dataset selection and versioning, the definition of positive interactions, filtering rules, and evaluation splits can materially change the data available to a model and thus its measured performance. When these choices are implicit, difficult to reproduce, or tied to a specific software ecosystem, apparently comparable experiments may instead evaluate different tasks. This issue contributes to the reproducibility concerns repeatedly identified in recommender-systems research [5, 6, 7, 8, 9, 10, 11, 12, 13].

Existing frameworks have substantially improved the implementation and evaluation of recommender algorithms [14, 15, 16, 17]. However, most provide end-to-end, framework-specific pipelines. Their data-management capabilities differ in coverage, semantics, and output formats, making it difficult to repeat the same transformations while comparing models implemented in distinct frameworks. DataRec addresses this gap as a focused, modular library: it does not replace recommendation frameworks, but provides a common layer for preparing, documenting, validating, and exporting recommendation data.

The original work makes three contributions: (i) an analysis of current data-handling practices in recommendation research; (ii) a comparison of the data-related functionality offered by widely used

Proceedings of the 16th Italian Information Retrieval Workshop, IIR 2026, November 12–14, 2026, Rome, Italy

⋆This paper is an extended abstract of the article "DataRec: A Python Library for Standardized and Reproducible Data Management in Recommender Systems," published in the Proceedings of ACM SIGIR 2025 [1].

†Work done while at Politecnico di Bari.

alberto.mancino@poliba.it (A. C. M. Mancino); bufi.salvatore.work@gmail.com (S. Bufi); angela.difazio@poliba.it (A. D. Fazio); antonio.ferrara@poliba.it (A. Ferrara); dmalitesta@luiss.it (D. Malitesta); claudio.pomo@poliba.it (C. Pomo); tommaso.dinoia@poliba.it (T. D. Noia)

0000-0001-8027-9475 (A. C. M. Mancino); 0009-0001-6979-1715 (S. Bufi); 0009-0007-3238-4327 (A. D. Fazio); 0000-0002-1921-8304 (A. Ferrara); 0000-0003-2228-0333 (D. Malitesta); 0000-0001-5206-3909 (C. Pomo); 0000-0002-0939-5462 (T. D. Noia)

© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

frameworks; and (iii) DataRec, an open-source library implementing traceable and interoperable data-management routines. Source code, documentation, and examples are available at https://github.com/sisinflab/DataRec.

2. Why data management needs standardization

To ground the library in current practice, we analyzed 55 papers published between 2020 and 2024, selected from surveys spanning graph-based, contrastive, temporal, reinforcement-learning, and generative recommendation. The survey reveals substantial diversity in dataset provenance, preprocessing, and partitioning. Diversity is valuable because recommendation research addresses many domains; the concern is that the information needed to recreate a particular data pipeline is frequently incomplete.

Across the surveyed studies, we identified 79 distinct datasets and 179 dataset usages. Only 35.2% of those usages explicitly referenced the original data source. In 32.4% of cases, authors referred to a copied or already processed version without clearly documenting its provenance; 16.8% cited only an original dataset paper or another scholarly paper; and 15.6% were irretrievable because the source was absent or unavailable. These findings motivate explicit source references, version awareness, and executable preprocessing histories [18].

These observations have implications beyond the practical inconvenience of downloading a file. A dataset name can refer to multiple releases, mirrors, subsets, or versions that have already undergone undisclosed filtering. For explicit-feedback data, the meaning of a rating changes after binarization; for temporal data, a split depends on timestamp parsing and ordering; and for sparse data, an iterative k-core can remove a different population from a one-pass filter. Without recording such decisions, it is difficult to establish whether a result can be replicated from the claimed input or whether two reported results evaluate comparable tasks.

Filtering and splitting also vary considerably. Common operations include binarizing explicit ratings, removing low-activity users or items through k-core filtering, discarding interactions by rating or time, and partitioning data randomly, temporally, or with predefined splits. Each choice may be justified by the task, but should be represented precisely and rerunnable. A complementary inspection of established frameworks, including ClayRS, Cornac, DaisyRec, Elliot, LensKit, RecBole, ReChorus, Recommenders, and RecPack, showed heterogeneous support for these operations and limited direct interoperability [19, 20, 21, 22]. DataRec was designed to provide a shared implementation of the most recurrent practices while preserving the ability to use a preferred modelling framework.

The survey does not prescribe a universal evaluation protocol. Instead, it identifies recurrent alternatives that should have clear semantics, exposed parameters, and a reusable implementation. For example, a temporal split may be needed to avoid information leakage in a next-item setting, whereas a random split can be appropriate for another task. Standardization in DataRec means that the chosen procedure is explicit and can be rerun; it does not mean that every study must make identical methodological choices.

3. DataRec design

Figure 1 summarizes the library. The central `DataRec` object represents a recommendation dataset as a tabular collection of user–item interactions, optionally enriched with ratings and timestamps. It supplies descriptive statistics relevant to recommendation data, including shape, density, space size, Gini coefficients, average interactions, and popularity groups. Three modules operate on this common representation: input/output, processing, and splitting.

3.1. Data access, formats, and interoperability

The I/O module reads and writes the common forms of recommendation data: tabular interaction files, inline user histories for implicit feedback, and JSON data with interaction metadata. It can therefore



Figure 1: DataRec architecture. A `DataRec` object connects I/O, processing, and splitting modules.

convert heterogeneous raw representations into a consistent internal schema. DataRec also provides built-in access to frequently used datasets identified in the survey. For each supported version, it preserves a reference to the source whenever possible and publishes a checksum that validates the downloaded resource. This makes dataset acquisition part of the documented experiment rather than an untracked preliminary step.

Processed datasets can be exported in formats compatible with ClayRS, Cornac, DaisyRec, Elliot, LensKit, RecBole, ReChorus, Microsoft Recommenders, and RecPack. These exporters let researchers apply the same dataset preparation before evaluating implementations from different ecosystems. The goal is not to impose one modelling stack, but to make the data entering each stack comparable and inspectable.

3.2. Processing and splitting

The processing module exposes the recurrent transformations found in the literature. A binarizer converts explicit ratings to implicit feedback according to a configurable threshold. k-core operations remove users and/or items with too few interactions; iterative variants repeat the procedure until the criterion is met or a specified iteration limit is reached. Filters can retain interactions by rating, using a fixed, global, or user-specific threshold, or by time. The operations are composable, so a pipeline can state not merely that a dataset was "preprocessed," but exactly how.

The order of operations can change the resulting dataset. Applying a rating threshold before a k-core may retain a different set of users and items from applying the two operations in the opposite order. Likewise, an iterative k-core may require several rounds because removing low-activity items can make users fall below the threshold, and vice versa. DataRec records the operation and its parameters at every step, allowing a reader to distinguish these alternatives. The transformed object can also be inspected through descriptive measures, including density, sparsity, popularity groups, Gini coefficients, and user/item interaction distributions.

The splitting module supports temporal, random, and pre-computed partitions. Temporal splitting

can use a cutoff or hold out recent interactions; random splitting supports hold-out, repeated hold-out, and cross-validation. Where appropriate, the logic is user-stratified, so each user's history is partitioned independently. These alternatives accommodate distinct evaluation protocols without hiding their choices in ad-hoc scripts, particularly because no universally accepted splitting method exists [23, 24].

4. Reproducibility as an output

DataRec treats a data pipeline as a first-class research artifact. Each operation applied to a `DataRec` object is appended to its history together with the operation name and parameters. Stochastic procedures expose a random seed, and the state after each step is associated with a checksum. The complete history can be exported as a configuration file and replayed to reconstruct the pipeline, while checksums allow the generated intermediate and final data to be verified.

This mechanism supports practical questions that are otherwise difficult to answer: Which exact MovieLens version was used? Was binarization applied before filtering? Did a random split use the same seed? Did an exported dataset match the intended one? Making these answers machine-readable reduces accidental discrepancies, facilitates replication, and provides a compact description that can accompany a paper or repository.

The history mechanism has two complementary roles. First, it is descriptive: it records the provenance of a derived dataset in a form that can be inspected and shared. Second, it is verificatory: a replayed pipeline can be checked against the checksum associated with each intermediate state. This is valuable when an external source changes, an implementation is updated, or a dataset is moved across machines. A mismatch does not silently propagate into model training; it signals that the reconstructed input differs from the one described by the original pipeline.

By exporting this information as a configuration file, DataRec lowers the cost of reporting data preparation. Authors can supplement a paper with an executable artifact rather than relying on prose alone. Reviewers and other researchers can inspect the exact chain of loading, processing, splitting, and export operations. This does not eliminate all sources of experimental variance, such as model implementations, hardware, and training choices, but it isolates and documents a foundational source of variance that is often overlooked.

5. Conclusion and outlook

DataRec focuses on a foundational but often under-specified stage of recommender-system experimentation. It combines traceable dataset access, a unified data representation, configurable transformations and splits, replayable histories, and framework-specific exporters. The library was informed by observed practices rather than by a single evaluation protocol: it enables researchers to make their methodological choices explicit and reproduce them across projects and frameworks.

As an open project, DataRec can evolve with new datasets, processing strategies, and recommendation paradigms. We invite the community to extend the library.

Acknowledgments

The authors acknowledge partial support from the following projects: "LIFE Project: the itaLian system wide Frailty nEtwork" (local code: T2-AN-12; CUP: D93C22000640001); "Research Actions for Reducing the Impact on Agricultural and Natural Ecosystems of the Harmful Plant Pathogen Xylella fastidiosa" (REACH-XY; CUP: B93C22001920001); "Advanced Cooperative SOC" (ACSOC), funded by the European Union's Digital Europe Programme (Grant Agreement No. 101190248); Grifo Multimedia – "LUtech Campus Ecosystem – LUCE" (Application Code: 22ROJB5; CUP: B95I25000160007); and "AURA – Augmented Unified Resource for Agents" (Applicant: KONECTA ITALIA S.P.A.; Application Code: CDP001572). The authors also acknowledge ISCRA for awarding access to the LEONARDO supercomputer, hosted by CINECA (Italy).

Declaration on Generative AI

During the preparation of this work, the authors used OpenAI ChatGPT (GPT-4.5) for language revision and editing. The authors reviewed and revised the AI-assisted content as needed and take full responsibility for the final manuscript.

References

[1] A. C. M. Mancino, S. Bufi, A. D. Fazio, A. Ferrara, D. Malitesta, C. Pomo, T. D. Noia, Datarec: A python library for standardized and reproducible data management in recommender systems, in: SIGIR, ACM, 2025, pp. 3478–3487.

[2] C. Mallamaci, A. V. Petrov, A. C. M. Mancino, V. W. Anelli, T. D. Noia, C. Macdonald, Balancing accuracy and novelty with sub-item popularity, in: RecSys, ACM, 2025, pp. 1137–1141.

[3] S. Bufi, A. C. M. Mancino, A. Ferrara, D. Malitesta, T. D. Noia, E. D. Sciascio, KGUF: simple knowledge-aware graph-based recommender with user-based semantic features filtering, in: L. Boratto, D. Malitesta, M. Marras, G. Medda, C. Musto, E. Purificato (Eds.), Advances on Graph-Based Approaches in Information Retrieval - First International Workshop, IRonGraphs 2024, Glasgow, UK, March 24, 2024, Proceedings, volume 2197 of *Communications in Computer and Information Science*, Springer, 2024, pp. 41–59. URL: https://doi.org/10.1007/978-3-031-71382-8_4. doi:10.1007/978-3-031-71382-8_4.

[4] A. Ferrara, A. D. Fazio, A. C. M. Mancino, T. D. Noia, E. D. Sciascio, Enhancing utility in differentially private recommendation data release via exponential mechanism, in: ECIR (3), volume 15574 of *Lecture Notes in Computer Science*, Springer, 2025, pp. 34–51.

[5] M. F. Dacrema, P. Cremonesi, D. Jannach, Are we really making much progress? A worrying analysis of recent neural recommendation approaches, in: RecSys, ACM, 2019, pp. 101–109.

[6] Z. Sun, D. Yu, H. Fang, J. Yang, X. Qu, J. Zhang, C. Geng, Are we evaluating rigorously? benchmarking recommendation for reproducible evaluation and fair comparison, in: RecSys, ACM, 2020, pp. 23–32.

[7] M. F. Dacrema, S. Boglio, P. Cremonesi, D. Jannach, A troubling analysis of reproducibility and progress in recommender systems research, ACM Trans. Inf. Syst. 39 (2021) 20:1–20:49. URL: https://doi.org/10.1145/3434185. doi:10.1145/3434185.

[8] A. Bellogín, A. Said, Improving accountability in recommender systems research through reproducibility, User Model. User Adapt. Interact. 31 (2021) 941–977.

[9] E. Zangerle, C. Bauer, Evaluating recommender systems: Survey and framework, ACM Comput. Surv. 55 (2023) 170:1–170:38.

[10] M. Attimonelli, D. Danese, D. Malitesta, C. Pomo, G. Gassi, T. D. Noia, Ducho 2.0: Towards a more up-to-date unified framework for the extraction of multimodal features in recommendation, in: T. Chua, C. Ngo, R. K. Lee, R. Kumar, H. W. Lauw (Eds.), Companion Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, Singapore, May 13-17, 2024, ACM, 2024, pp. 1075–1078. URL: https://doi.org/10.1145/3589335.3651440. doi:10.1145/3589335.3651440.

[11] D. Malitesta, G. Gassi, C. Pomo, T. D. Noia, Ducho: A unified framework for the extraction of multimodal features in recommendation, in: A. El-Saddik, T. Mei, R. Cucchiara, M. Bertini, D. P. T. Vallejo, P. K. Atrey, M. S. Hossain (Eds.), Proceedings of the 31st ACM International Conference on Multimedia, MM 2023, Ottawa, ON, Canada, 29 October 2023- 3 November 2023, ACM, 2023, pp. 9668–9671. URL: https://doi.org/10.1145/3581783.3613458. doi:10.1145/3581783.3613458.

[12] D. Malitesta, E. Rossi, C. Pomo, T. D. Noia, F. D. Malliaros, Do we really need to drop items with missing modalities in multimodal recommendation?, in: E. Serra, F. Spezzano (Eds.), Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM 2024, Boise, ID, USA, October 21-25, 2024, ACM, 2024, pp. 3943–3948. URL: https://doi.org/10.1145/3627673.3679898. doi:10.1145/3627673.3679898.

[13] D. Malitesta, A. C. M. Mancino, C. Pomo, V. W. Anelli, A. Bellogín, T. D. Noia, On the impact of

graph neural networks in recommender systems: A topological perspective, Found. Trends Inf. Retr. (2027). In press.
[14] M. D. Ekstrand, Lenskit for python: Next-generation software for recommender systems experiments, in: CIKM, ACM, 2020, pp. 2999–3006.
[15] S. Graham, J. Min, T. Wu, Microsoft recommenders: tools to accelerate developing recommender systems, in: RecSys, ACM, 2019, pp. 542–543.
[16] L. Michiels, R. Verachtert, B. Goethals, Recpack: An(other) experimentation toolkit for top-n recommendation using implicit feedback data, in: RecSys, ACM, 2022, pp. 648–651.
[17] C. Wang, M. Zhang, W. Ma, Y. Liu, S. Ma, Make it a chorus: Knowledge- and time-aware item modeling for sequential recommendation, in: SIGIR, ACM, 2020, pp. 109–118.
[18] V. Paparella, A. C. M. Mancino, A. Ferrara, C. Pomo, V. W. Anelli, T. D. Noia, Knowledge graph datasets for recommendation, in: KaRS@RecSys, volume 3560 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2023, pp. 109–117.
[19] P. Lops, M. Polignano, C. Musto, A. Silletti, G. Semeraro, Clayrs: An end-to-end framework for reproducible knowledge-aware recommender systems, Inf. Syst. 119 (2023) 102273.
[20] A. Salah, Q. Truong, H. W. Lauw, Cornac: A comparative framework for multimodal recommender systems, J. Mach. Learn. Res. 21 (2020) 95:1–95:5.
[21] V. W. Anelli, A. Bellogín, A. Ferrara, D. Malitesta, F. A. Merra, C. Pomo, F. M. Donini, T. D. Noia, Elliot: A comprehensive and rigorous framework for reproducible recommender systems evaluation, in: SIGIR, ACM, 2021, pp. 2405–2414.
[22] W. X. Zhao, S. Mu, Y. Hou, Z. Lin, Y. Chen, X. Pan, K. Li, Y. Lu, H. Wang, C. Tian, Y. Min, Z. Feng, X. Fan, X. Chen, P. Wang, W. Ji, Y. Li, X. Wang, J. Wen, Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms, in: CIKM, ACM, 2021, pp. 4653–4664.
[23] Z. Meng, R. McCreadie, C. Macdonald, I. Ounis, Exploring data splitting strategies for the evaluation of recommendation models, in: RecSys, ACM, 2020, pp. 681–686.
[24] A. Sun, Take a fresh look at recommender systems from an evaluation standpoint, in: SIGIR, ACM, 2023, pp. 2629–2638.